

程序报告

学号：2312331

姓名：王一诺

一、问题重述

(简单描述对问题的理解，从问题中抓住主干，必填)

(1) 黑白棋问题：黑白棋 (Reversi)，也叫苹果棋，翻转棋，是一个经典的策略性游戏。一般棋子双面为黑白两色，故称“黑白棋”。因为行棋之时将对方棋子翻转，则变为己方棋子，故又称“翻转棋”。棋子双面为红、绿色的称为“苹果棋”。它使用 8x8 的棋盘，由两人执黑子和白子轮流下棋，最后子多方为胜方。

(2) 游戏规则：

1. 黑方先行，双方交替下棋。
2. 一步合法的棋步包括：在一个空格处落下一个棋子，并且翻转对手一个或多个棋子；新落下的棋子必须落在可夹住对方棋子的位置上，对方被夹住的所有棋子都要翻转过来，可以是横着夹，竖着夹，或是斜着夹。夹住的位置上必须全部是对手的棋子，不能有空格；一步棋可以在数个（横向，纵向，对角线）方向上翻棋，任何被夹住的棋子都必须被翻转过来，棋手无权选择不翻某个棋子。
3. 如果一方没有合法棋步，也就是说不管他下到哪里，都不能至少翻转对手的一个棋子，那他这一轮只能弃权，而由他的对手继续落子直到他有合法棋步可下。
4. 如果一方至少有一步合法棋步可下，他就必须落子，不得弃权。
5. 棋局持续下去，直到棋盘填满或者双方都无合法棋步可下。
6. 如果某一方落子时间超过 1 分钟 或者 连续落子 3 次不合法，则判该方失败。

(3) 实验要求：使用『蒙特卡洛树搜索算法』实现 miniAlphaGo for Reversi。使用 Python 语言。算法部分需要自己实现，不要使用现成的包、工具或者接口。

二、设计思想

(所采用的方法，有无对方法加以改进，该方法有哪些优化方向（参数调整，框架调整，或者指出方法的局限性和常见问题），伪代码，理论结果验证等... 思考题，非必填)

采用蒙特卡洛树搜索（简称 MCTS）方法，主要有以下四个重要的步骤：选择、扩展、模拟、反向传播。

1. 先设计一个计算 UCB1 值的函数，主要流程为：遍历所有合法动作，计算总访问次数 rec_sum ；再次遍历动作，计算每个动作的 UCB 值： $score_tmp = (scr/rec) + C * \sqrt{\ln(rec_sum)/rec}$ ；其中 scr 为累计得分， rec 为访问次数， C 为探索参数。最后返回 UCB 值最高的动作。

2. 选择阶段，设计一个 Select 函数选择未完全扩展的节点或继续深入树。大致流程为：检查当前节点的所有子节点是否均被访问过 ($all_explored$)；若全访问过，用 UCB1 选择子节点并继续深入，否则，随机选择未访问的子节点返回以扩展。

3. 扩展阶段：设计一个 Expand 函数扩展新节点并进行模拟。大致流程为：若选择的节点不是终止节点，则随机扩展它的一个为扩展过的后继节点。调用 Simulate 进行随机模拟，得到分数差 scr_diff ；初始化新节点的访问次数为 1，并根据当前玩家调整分数差。

- 4.模拟阶段：设计一个 **Simulate** 函数随机模拟扩展搜索树至终局。
- 5.反向传播：设计一个 **BackPropagate** 函数回溯更新路径上的节点数据，即根据上一环节的模拟结果回溯更新模拟路径上的奖励值和被访问次数。

三、代码内容

（能体现解题思路的主要代码，有多个文件或模块可用多个"===="隔开，**必填**）

```
=====
import math
import random
import datetime
import numpy as np
from copy import deepcopy
class AIPlayer:
    """
    AI 玩家
    """
    def UCB1(self, color, board_in):
        """
        根据 UCB1 公式选择最优子节点
        color: 当前节点对应的玩家颜色
        board_in: 当前棋盘状态
        return: 最优落子位置
        """
        board = deepcopy(board_in)
        score_act = None
        act = None
        action_list = list(board.get_legal_actions(color))
        rec_sum = 0 # 当前节点的总访问次数（用于计算 UCB）
        # 计算当前节点的总访问次数
        for action in action_list:
            play_board = deepcopy(board)
            play_board._move(action, color)
            tmp_key = tuple(np.ravel(play_board._board))
            if self.rec.get((color, tmp_key)):
                rec_sum += self.rec.get((color, tmp_key))
        # 计算每个子节点的 UCB 值并选择最优
        for action in action_list:
            play_board = deepcopy(board)
            play_board._move(action, color)
            tmp_key = tuple(np.ravel(play_board._board))
            # UCB1 公式： exploitation + exploration
            exploitation = self.scr.get((color, tmp_key), 0) / self.rec.get((color, tmp_key), 1)
            exploration = self.C * math.sqrt(math.log(rec_sum + 1) / self.rec.get((color, tmp_key),
1))
```

```

        score_tmp = exploitation + exploration
        # 更新最优动作
        if score_act is None or score_act < score_tmp:
            score_act, act = score_tmp, action
    return act

def Select(self, board):
    """
    选择阶段：从根节点开始选择子节点，直到遇到未完全扩展的节点或叶子节点
    board: 当前棋盘状态
    return: (color, act, tmpkey) 或 (None, None, None)（无合法动作时）
    """
    color = self.color
    while True:
        action_list = list(board.get_legal_actions(color))
        if not action_list:
            return None, None, None # 无合法动作

        all_explored = True # 标记是否所有子节点均已访问
        non_vis_son = [] # 未访问的子节点列表
        rec_sum = 0 # 已访问子节点的总访问次数
        # 检查每个子节点的访问状态
        for action in action_list:
            play_board = deepcopy(board)
            play_board._move(action, color)
            tmp_key = tuple(np.ravel(play_board._board))

            if not self.rec.get((color, tmp_key)):
                all_explored = False
                non_vis_son.append((action, tmp_key))
            else:
                rec_sum += self.rec.get((color, tmp_key))
        # 根据子节点状态决定下一步
        if all_explored:
            # 所有子节点已访问：选择 UCB1 最优的子节点继续深入
            act = self.UCB1(color, board)
            if act is None:
                return None, None, None
        else:
            # 存在未访问子节点：随机选择一个用于扩展
            act, tmp_key = random.choice(non_vis_son)
            board._move(act, color)
            return color, act, tmp_key
    # 继续向子节点深入

```

```

        board._move(act, color)
        tmp_key = tuple(np.ravel(board._board))
        self.vis.add((color, tmp_key))
        color = "X" if color == "O" else "O" # 切换玩家

def Expand(self, board, color, act, tmpkey):
    """
    扩展阶段：为新节点执行模拟并初始化统计信息
    board: 当前棋盘状态
    color: 最后一个落子的玩家颜色
    act: 最后一个落子位置
    tmpkey: 当前棋盘状态的唯一键
    return: 调整后的分数差
    """

    game_state, scr_diff = self.Simulate(board, color)
    # 初始化新节点的统计数据
    self.rec[(color, tmpkey)] = 1
    # 调整分数差为 AI 视角 (AI 得分 - 对手得分)
    if (game_state == 0 and self.color == "O") or (game_state == 1 and self.color == "X"):
        scr_diff = -scr_diff
    scr_diff *= 0.4 # 分数差调整系数
    # 根据当前节点玩家颜色记录分数
    if color == self.color:
        self.scr[(color, tmpkey)] = scr_diff
    else:
        self.scr[(color, tmpkey)] = -scr_diff
    return scr_diff

def Simulate(self, board, player):
    """
    模拟阶段：随机模拟游戏至终局
    board: 当前棋盘状态
    player: 最后一个落子的玩家
    return: (赢家, 分数差)
    """

    while True:
        player = "X" if player == "O" else "O" # 切换玩家
        legal_actions = list(board.get_legal_actions(player))
        # 处理无合法动作的情况
        if not legal_actions:
            if self.game_over(board):
                return board.get_winner() # 游戏结束返回赢家
            else:
                continue # 弃权, 跳过回合

```

```

        # 随机选择动作进行模拟
        action = random.choice(legal_actions)
        board._move(action, player)
        # 检查游戏是否结束
        if self.game_over(board):
            return board.get_winner()

def BackPropagate(self, scr_diff):
    """
    反向传播：更新搜索路径上的节点统计数据
    scr_diff: 当前模拟的分数差（AI 视角）
    """
    for (color, key) in self.vis:
        self.rec[(color, key)] += 1 # 更新访问次数
        # 根据节点颜色更新累计得分
        if color == self.color:
            self.scr[(color, key)] += scr_diff
        else:
            self.scr[(color, key)] -= scr_diff

def game_over(self, board):
    """
    检查游戏是否结束
    board: 棋盘状态
    return: bool
    """
    return len(list(board.get_legal_actions("X"))) == 0 and \
           len(list(board.get_legal_actions("O"))) == 0

def __init__(self, color):
    """
    玩家初始化
    :param color: 下棋方，'X' - 黑棋，'O' - 白棋
    """
    self.color = color # 当前玩家颜色
    self.C = 1.4 # UCB1 探索系数
    self.rec = {} # 记录节点访问次数：(color, board_key): count
    self.scr = {} # 记录节点累计得分：(color, board_key): score
    self.vis = set() # 记录当前搜索路径中的节点

def get_move(self, board):
    """
    根据当前棋盘状态获取最佳落子位置
    board: 棋盘
    """

```

```
:return: action 最佳落子位置, e.g. 'A1'
"""

if self.color == 'X':
    player_name = '黑棋'
else:
    player_name = '白棋'
print("请等一会, 对方 {}-{} 正在思考中...".format(player_name, self.color))

start_time = datetime.datetime.now()
count = 0
# 主搜索循环 (时间限制 30 秒或最多 10000000 次迭代)
while True:
    count += 1
    current_time = datetime.datetime.now()
    if (current_time - start_time).seconds > 30 or count > 10000000:
        break
    board_1 = deepcopy(board)
    self.vis = set() # 清空当前搜索路径
    # 选择阶段
    color, act, tmpkey = self.Select(board_1)
    if color is None:
        continue # 无合法动作时跳过
    # 扩展和模拟阶段
    scr_diff = self.Expand(board_1, color, act, tmpkey)
    # 反向传播
    self.BackPropagate(scr_diff)
# 返回 UCB1 最优的根节点子节点
action = self.UCB1(self.color, board)
return action
```

四、实验结果

(实验结果, 必填)

测试 1: 高级难度, 黑棋先手

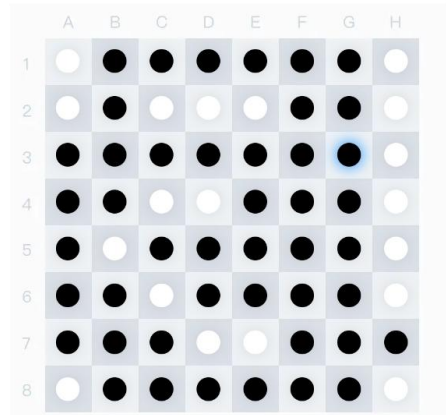
[测试详情](#) [展示棋盘](#) 

测试点	状态	时长	结果
对手对弈		995s	黑棋获胜, 领先棋子数: 26

接口测试

✓ 接口测试通过。

用例测试



棋局胜负: 黑棋赢

先后手: 黑棋先手

棋局难度: 高级

当前棋子: 黑棋

当前坐标: G3

可见我们领先了对方 26 子，取胜；
测试 2：高级难度，白棋后手

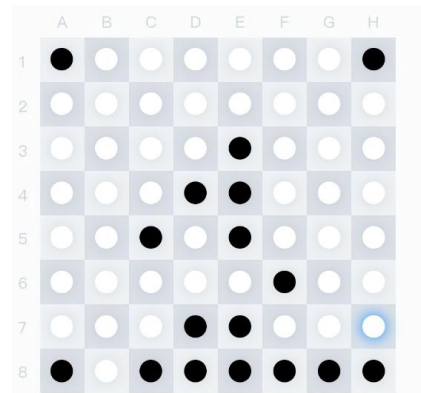
接口测试

✓ 接口测试通过。

用例测试

展示棋盘 ▾

测试点	状态	时长	结果
对手对弈	✓	951s	白棋获胜, 领先棋子数: 30



棋局胜负: 白棋赢

先后手: 白棋后手

棋局难度: 高级

当前棋子: 白棋

当前坐标: H7

可见我们领先了对方 30 子，取胜。

五、总结

（自评分析（是否达到目标预期，可能改进的方向，实现过程中遇到的困难，从哪些方面可以提升性能，模型的超参数和框架搜索是否合理等），**思考题，非必填**）

1. 实验的关键在于理解和实现蒙特卡洛树搜索的四阶段：选择、扩展、模拟和反向传播；
2. 除核心逻辑外，还要注意代码细节，比如要限制在什么时候触发 Expand? UCB1 计算时是否会出现除零错误？主搜索循环的终止条件设定？等等等等。